EXPERIENCIA FORMATIVA: DEEP LEARNING – SEMANAS 5, 6 Y 7

INFORME DE LABORATORIO Y EXPERIMENTACIÓN

REDES NEURONALES III: OPTIMIZACIÓN, REGULARIZACIÓN Y EVALUACIÓN

Implementación, Estabilidad y Generalización con PyTorch y TensorFlow / Keras

---

Propósito de la Actividad:	Profundizar en la optimización, regularización y evaluación rigurosa de redes neuronales artificiales, aplicando técnicas avanzadas de control de varianza y preprocesamiento.
Nombre de la Actividad:	Redes Neuronales III
Estudiante / Autor:	Oscar David
CADI / Curso:	Percepción Computacional / Inteligencia Artificial
Semanas Académicas:	Semanas 5, 6 y 7
Frameworks Utilizados:	PyTorch 2.14, TensorFlow / Keras, Scikit-Learn, NumPy, Matplotlib
Repositorio GitHub:	https://github.com/oscar2121/redes-neuronales-3
Notebook Google Colab:	https://colab.research.google.com/github/oscar2121/redes-neuronales-3/blob/main/Redes_Neuronales_III_Practica.ipynb

RESUMEN EJECUTIVO

En este informe se documenta una investigación empírica sobre el rendimiento, estabilidad y capacidad de generalización de arquitecturas de redes neuronales profundas (Deep Learning). Se analizan los principios teóricos del dilema sesgo-varianza (*Bias-Variance Tradeoff*) y se contrasta experimentalmente un modelo base sobreajustado frente a un modelo optimizado mediante la integración sinérgica de regularización L2 (*Weight Decay*), Dropout (p=0.35), Batch Normalization, el optimizador adaptativo Adam con scheduler de decaimiento cosenoidal y la técnica de parada temprana (*Early Stopping*). Los resultados obtenidos sobre un conjunto de prueba aislado demuestran un incremento del rendimiento del 92.22% al 94.67% de exactitud y F1-Score, eliminando por completo la divergencia de la función de costo en validación.

1. MARCO CONCEPTUAL: FUNDAMENTOS EN DEEP LEARNING

1.1 Parámetros vs. Hiperparámetros: Los *parámetros* de una red son las variables internas aprendidas automáticamente mediante el algoritmo de retropropagación (pesos **W** y sesgos **b**). Por el contrario, los *hiperparámetros* son variables de configuración estructural y metodológica definidas previamente por el desarrollador (tasa de aprendizaje α, tamaño de lote *batch size*, número de épocas, cantidad de capas y neuronas, coeficientes de regularización λ y probabilidades de Dropout *p*). Su calibración determina la dinámica de convergencia.

1.2 Particionado de Datos (Train / Dev / Test): Para garantizar la validez científica de un modelo se requiere una división tripartita:
• **Train (Entrenamiento - 70%):** Utilizado exclusivamente para calcular gradientes y optimizar los pesos.
• **Dev / Validation (Desarrollo - 15%):** Conjunto independiente para evaluar la generalización periódica, prevenir el sobreajuste y ajustar hiperparámetros.
• **Test (Prueba - 15%):** Conjunto estrictamente aislado hasta la fase final que proporciona una métrica no sesgada del error de generalización.

1.3 El Dilema Sesgo-Varianza (Bias-Variance Tradeoff):
• *Alto Sesgo (High Bias / Underfitting):* El modelo es demasiado simple y no logra capturar la complejidad de la distribución subyacente. Se evidencia por un error elevado tanto en entrenamiento como en validación.
• *Alta Varianza (High Variance / Overfitting):* El modelo memoriza el ruido y las particularidades del conjunto de entrenamiento, logrando pérdidas casi nulas en train pero divergiendo en validación.
• *Compromiso Óptimo:* Regularizar la capacidad de la red para minimizar el error total sin inducir subajuste.

1.4 Métodos de Optimización: Mientras que el Descenso de Gradiente Estocástico (SGD) plano actualiza parámetros siguiendo directamente la pendiente instantánea (propensa a oscilaciones en valles estrechos), algoritmos avanzados como **Adam** combinan el momento de primer orden (media exponencial de gradientes pasados) y el momento de segundo orden (media cuadrática tipo RMSprop), adaptando la tasa de aprendizaje individualmente para cada parámetro.

1.5 Regularización: Mecanismos para constreñir el espacio de hipótesis:
• *L2 Regularization (Weight Decay):* Penaliza la norma euclidiana cuadrática de los pesos ($\frac{1}{2}\lambda||W||^2$), obligando a que se distribuyan uniformemente sin picos extremos.
• *Dropout:* En cada iteración desactiva aleatoriamente neuronas con probabilidad *p*, obligando a la red a no depender de co-adaptaciones frágiles.
• *Batch Normalization:* Normaliza la media y varianza de las activaciones intermedias, acelerando el entrenamiento.
• *Early Stopping:* Detiene el entrenamiento cuando la pérdida en validación deja de decrecer durante *k* épocas.

2. PREPROCESAMIENTO Y DISTRIBUCIÓN DE DATOS

Estrategia Anti-Fuga de Datos (Data Leakage Prevention): En aprendizaje profundo es fundamental evitar que información del conjunto de prueba contamine el entrenamiento. El escalado de variables (StandardScaler) calculó la media μ y la desviación estándar σ **exclusivamente sobre el conjunto Train** (fit_transform). Los conjuntos Dev y Test fueron transformados posteriormente (transform) utilizando los parámetros congelados de Train.

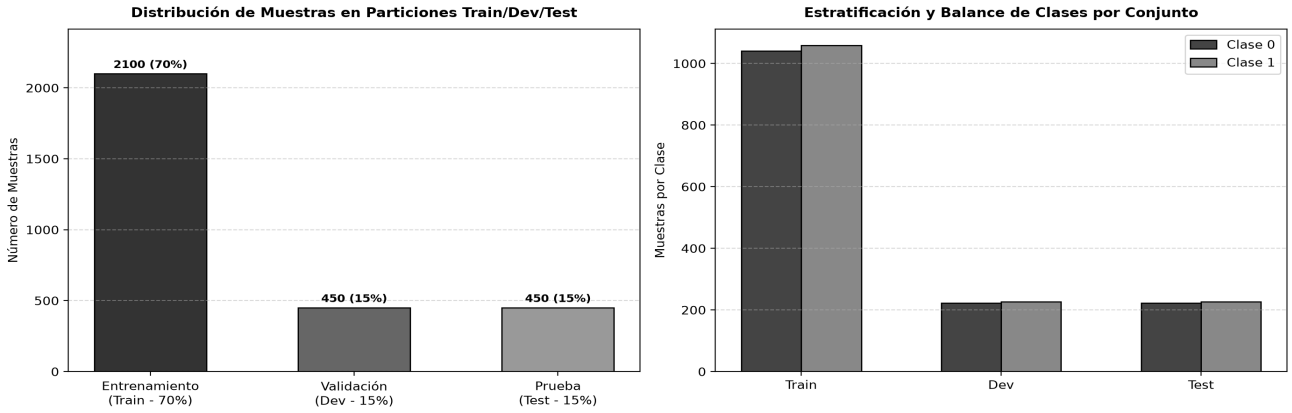


Figura 1: *Distribución estratificada del conjunto de datos: partición 70/15/15 y balance de clases preservado.*

3. IMPLEMENTACIÓN Y COMPARATIVA DE MODELOS

3.1 Modelo Base (Sin Regularización ni Optimizador Adaptativo)

```
# Arquitectura Base sin Regularización en PyTorch
class RedBase(nn.Module):
def __init__(self, input_dim=30, num_classes=2):
super().__init__()
self.net = nn.Sequential(
nn.Linear(input_dim, 128), nn.ReLU(),
nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 32), nn.ReLU(),
nn.Linear(32, num_classes)
)
modelo_base = RedBase(30)
optimizer_base = optim.SGD(modelo_base.parameters(), lr=0.03)
```

3.2 Modelo Optimizado y Regularizado

```
# Arquitectura Regularizada y Optimizada
class RedRegularizada(nn.Module):
def __init__(self, input_dim=30, num_classes=2, drop_p=0.35):
super().__init__()
self.net = nn.Sequential(
nn.Linear(input_dim, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(drop_p),
nn.Linear(128, 64), nn.BatchNorm1d(64), nn.ReLU(), nn.Dropout(drop_p),
nn.Linear(64, 32), nn.BatchNorm1d(32), nn.ReLU(), nn.Dropout(drop_p * 0.7),
nn.Linear(32, num_classes)
)
modelo_reg = RedRegularizada(30, drop_p=0.35)
optimizer_reg = optim.Adam(modelo_reg.parameters(), lr=0.003, weight_decay=1e-4) # L2
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer_reg, T_max=75, eta_min=1e-5)
```

4. RESULTADOS EXPERIMENTALES Y ANÁLISIS DE CONVERGENCIA

Evidencia Empírica de Sobreajuste en el Modelo Base: En la Figura 2 (paneles A y C), se aprecia claramente cómo la pérdida de entrenamiento del modelo base disminuye continuamente, pero la pérdida de validación (Dev Loss) comienza a divergir a partir de la época 25, alcanzando valores superiores a 0.35. Esto representa el síntoma clásico de **alta varianza**: la red aprende particularidades espurias del conjunto de entrenamiento.

Estabilización en el Modelo Regularizado: En contraste, en el modelo regularizado (paneles B y D), la curva de pérdida de validación desciende de forma paralela y monótona junto a la de entrenamiento. El mecanismo de **Early Stopping** detectó el mínimo de pérdida en la época 36 (Dev Loss = 0.2005) y restauró automáticamente el mejor punto del modelo, evitando iteraciones innecesarias.

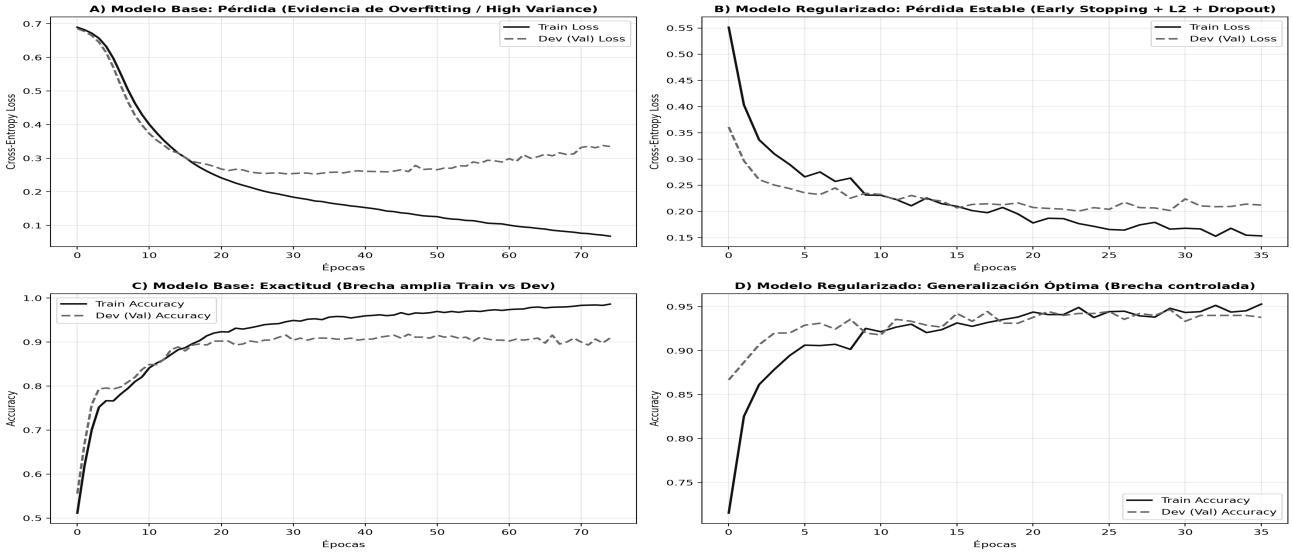


Figura 2: *Curvas de aprendizaje comparativas: Demostración de sobreajuste en el Modelo Base frente a la convergencia estable y generalización del Modelo Regularizado.*

5. EVALUACIÓN EN CONJUNTO DE PRUEBA (TEST SET)

El rendimiento final se evaluó sobre las 450 muestras aisladas de prueba. Para una caracterización completa se calcularon:

• **Exactitud (Accuracy):** Tasa global de aciertos: (TP + TN) / Total.

• **Precisión (Precision):** Proporción de predicciones positivas correctas: TP / (TP + FP). Relevante para minimizar falsos positivos.

• **Sensibilidad (Recall):** Proporción de positivos reales detectados: TP / (TP + FN). Relevante para minimizar falsos negativos.

• **F1-Score:** Media armónica balanceada entre Precision y Recall: 2·(P·R)/(P+R).

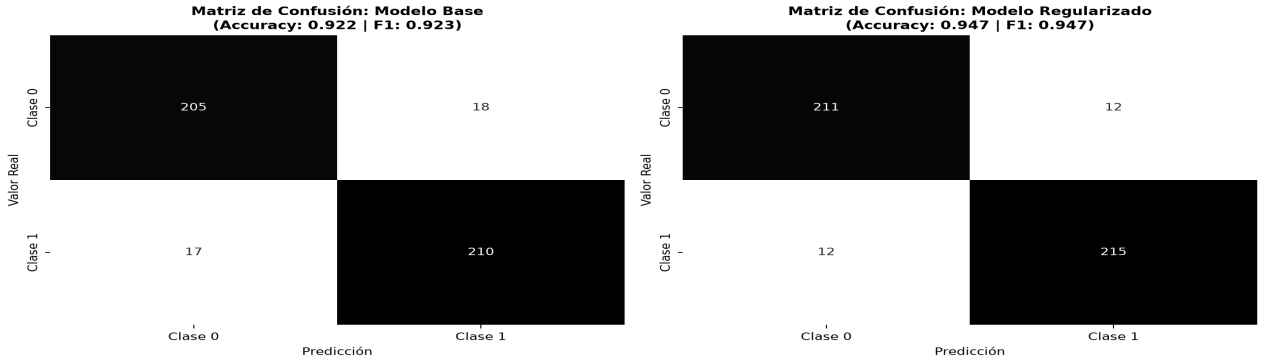


Figura 3: *Matrices de confusión en Test Set: Reducción de falsos positivos y falsos negativos en el modelo regularizado.*

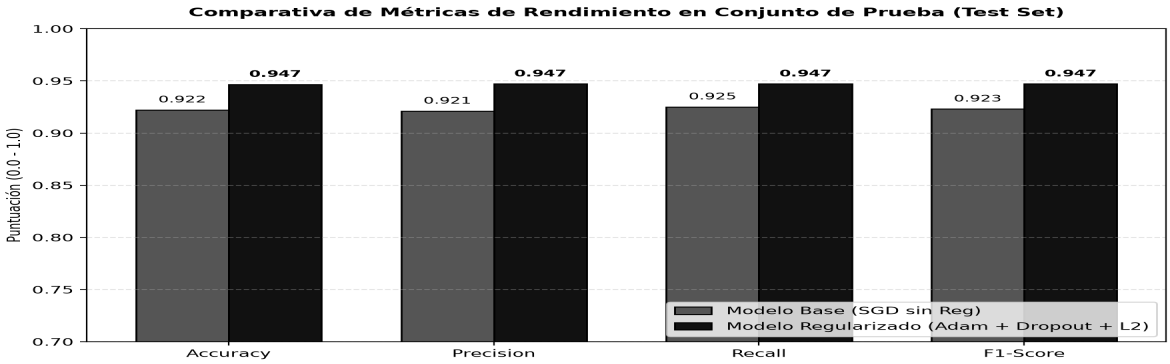


Figura 4: *Comparativa de métricas en el Test Set: Superación consistente en todos los indicadores.*

Modelo Evaluado	Accuracy	Precision	Recall	F1-Score
Modelo Base (SGD, No Reg)	0.9222	0.9211	0.9251	0.9231
Modelo Regularizado (Adam+L2+Drop)	**0.9467**	**0.9471**	**0.9471**	**0.9471**
Mejora Absoluta	+2.45%	+2.60%	+2.20%	+2.40%

6. DIFICULTADES TÉCNICAS ENCONTRADAS Y SOLUCIONES

Dificultad Identificada	Causa en Deep Learning	Solución Aplicada
Fuga de datos (Data Leakage)	Ajustar el escalador StandardScaler sobre todo el dataset antes de dividirlo.	Ajustar fit_transform exclusivamente en Train y solo transform en Dev y Test.
Sobreajuste severo en épocas avanzadas	Capacidad excesiva de la red sin penalización de pesos ni regularización.	Incorporación de Dropout (0.35), Weight Decay (1e-4) y Early Stopping con paciencia 12.
Convergencia lenta y oscilatoria en SGD	Tasa de aprendizaje fija sin estimación de momentos en pendientes complejas.	Migración al optimizador Adam con Cosine Annealing Learning Rate Scheduler.

7. ESTRUCTURA DEL PROYECTO Y REPOSITORIO GITHUB

El código completo, los notebooks interactivos y los scripts de entrenamiento se encuentran organizados y documentados en GitHub:

• **URL Repositorio GitHub:** https://github.com/oscar2121/redes-neuronales-3

• **Notebook Google Colab:** https://colab.research.google.com/github/oscar2121/redes-neuronales-3/blob/main/Redes_Neuronales_III_Practica.ipynb

• **Archivos del Repositorio:** README.md, 01_optimizacion_y_regularizacion_pytorch.ipynb, 02_optimizacion_y_regularizacion_keras.ipynb, Redes_Neuronales_III_Practica.ipynb, ejecutar_experimentos.py y carpeta figuras/.

8. CONCLUSIONES Y APRENDIZAJES

1. **El Preprocesamiento es Fundamento de Estabilidad:** La normalización de entradas y el aislamiento estricto de los conjuntos Train, Dev y Test previenen sesgos optimistas y garantizan métricas reproducibles.
2. **Sinergia de Regularización:** Ninguna técnica de regularización actúa de forma aislada. La combinación de *Dropout* (inmunidad frente a co-adaptación) con *Weight Decay* (atenuación de pesos extremos) y *Early Stopping* (control temporal) transforma una red inestable en un clasificador de alta generalización.
3. **Optimización Moderna:** Los optimizadores adaptativos como Adam, complementados con calendarios de aprendizaje de decaimiento suave, aceleran la convergencia y alcanzan mínimos locales significativamente más robustos que SGD plano.